

RAG: RETRIEVAL AUGMENTED GENERATION

Study notes

Mr. SUYOG BHATKHALKAR

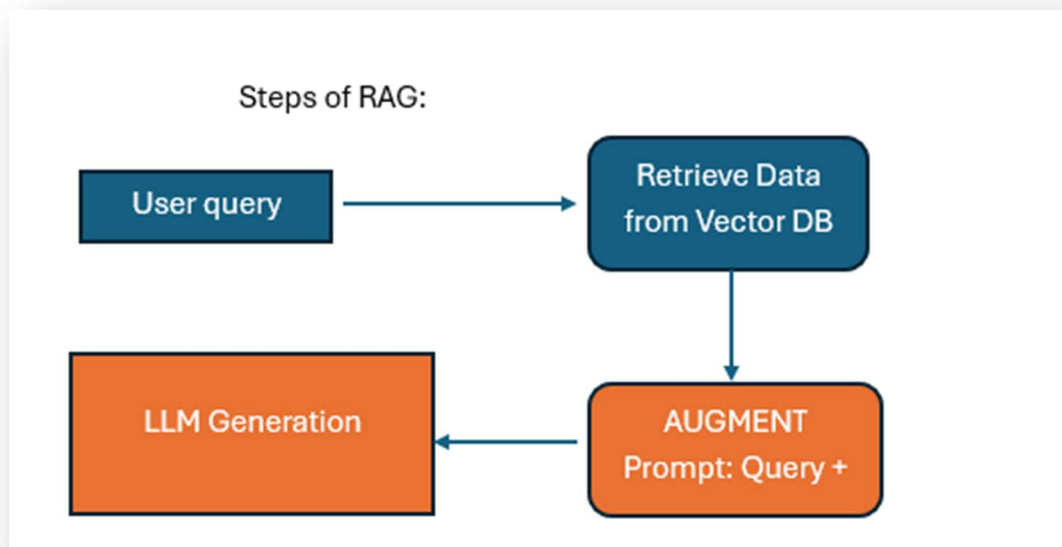
Contact me @ suyoginpune@gmail.com

What is RAG?

RAG Framework: Retrieval-Augmented Generation) is used to overcome token limitations of LLM, it provides

1. Eliminating "Hallucinations": By providing dynamic, domain specific useful context to the model.
2. Access to Private or Dynamic Data
3. Cost and Complexity Efficiency- Before RAG, the only way to inject new knowledge into a model was **Fine-Tuning** (retraining the model on new data).

How does it fit in LLM Pipeline?



1. The system takes the user's query, converts it into a vector embedding, and searches for a vector database to pull the most semantically relevant snippets of text from your documents.
2. **Augmentation:** The system stitches the user's original query and those retrieved text snippets together into a single, comprehensive prompt.
3. **Generation:** The LLM reads the combined prompt and uses the provided context to generate a highly accurate, grounded response.

What are the retrieval techniques?

1. Dense Vector Search: This search works on meaning/ semantics of user query. Example (Similar meaning "remote work" $\approx$ "work from home").
2. Parse Vector Search: This works on exact important words. Example: Exact keywords "Fast API", "AES-256", "Kafka"
3. Hybrid retrieval: Real-world systems rarely rely on single-modality retrieval. Hybrid pipelines compute a balanced vector space score alongside lexical frequency matching.

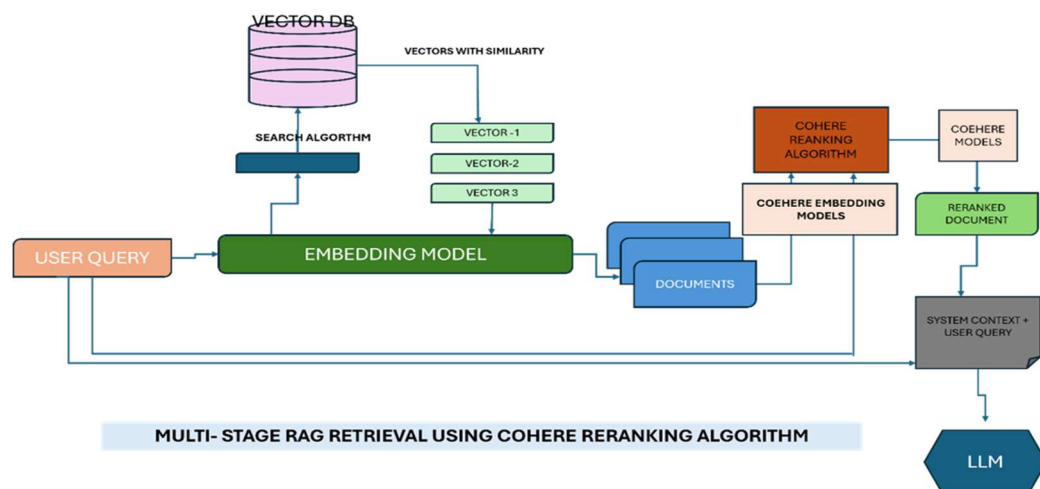
Reranking Algorithms or techniques

- a. Reciprocal Rank Fusion (RRF): RRF normalizes uniform scoring across completely different distributions. It evaluates the relative rank position of a document from both the sparse list and dense list, applying a regularized reciprocal formula:

$$RRF_Score(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

Where M represents the retrieval methods (Sparse and Dense), $r_m(d)$ is the explicit rank position of document d within method m, and k is a smoothing factor. The highest-scoring unified items are passed up to the generator.

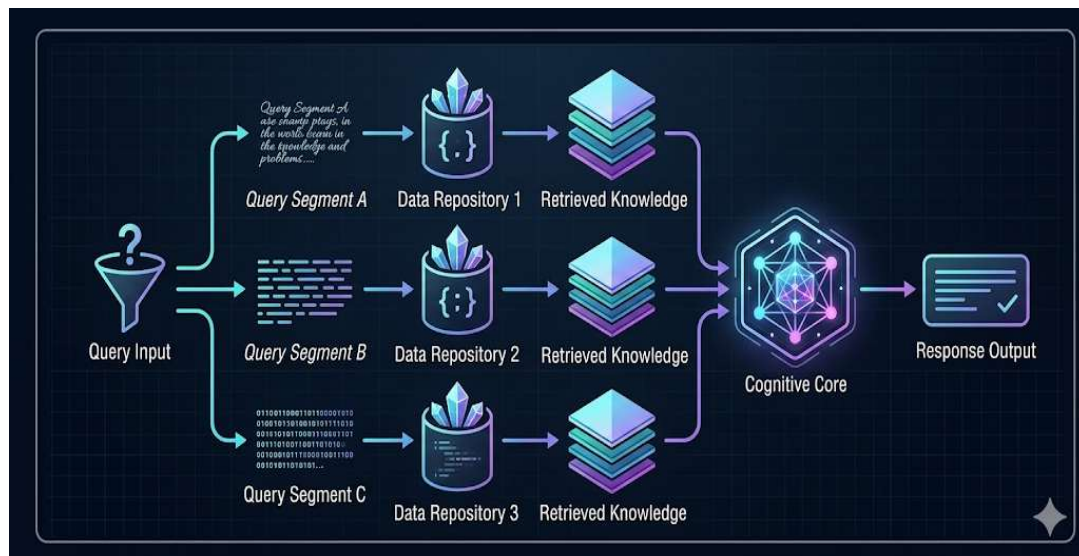
- b. Cohere reranking – Cohere represents multistep RAG retrieval pipeline with semantic / dense vector search .



Why cohere algorithm:

1. Standard RAG searches have vector math limitations where they work simply on the similarity (cosine) between the user question and documents.
2. Cohere Re rank uses a Cross-Encoder architecture. It takes the user query and a candidate document chunk, feeds them together into a massive transformer network, and allows full self-attention mechanisms to run across both strings simultaneously. Instead of checking if two vectors point in the same direction, it structurally evaluates exactly how well the document answers that specific query.

C. Multi-Query Retrieval : A user's single query can be insufficient to capture the full scope of information required to answer the query comprehensively. The multi-query retrieval strategy resolves this problem by instructing an LLM to generate multiple queries based on a user's initial query, executing a parallel retrieval of each query from the data source and then inserting the retrieved results as prompt context to generate a final model output.



D. Graph-Based Retrieval (Graph RAG): is an advanced AI pattern that combines traditional Retrieval-Augmented Generation (RAG) with **Knowledge Graphs (KGs)**. While baseline RAG relies purely on vector databases to find text chunks based on semantic similarity, Graph RAG indexes data as a network of **nodes** (entities like people, concepts, objects) and **edges** (the explicit relationships between them). This allows Large Language Models (LLMs) to reason across interconnected data points, maps, and hierarchies rather than just pulling isolated fragments of text. Standard vector-based RAG is excellent for localized

queries (e.g., "What is the return policy in document X?"), but it struggles with two specific types of complex queries: **Multi-hop reasoning**: Connecting non-obvious dots across multiple documents (e.g., "How is Project Alpha's supply chain vulnerability affected by the new regulatory updates in Europe?"). **Global/Aggregative questions**: Summarizing holistic themes across an entire dataset (e.g., "What are the main recurring technical challenges mentioned across all incident reports from last quarter?"). Vector databases miss the structural connections between ideas because text is chunked arbitrarily. Graph RAG solves this by explicitly tracking how entities relate to one another.

- E. Parent-child RAG (Hierarchical RAG)- Key value store: Instead of forcing a single chunk to serve both roles, the Parent-Child technique maps a hierarchical relationship inside your data layers. Raw documents are ingested and split into large, coherent blocks called Parents (or left as whole files). These are saved in a standard document store (e.g., an in-memory map or NoSQL database). Each parent is further sliced into granular, overlapping fragments called Children. Only the small child chunks are embedded and indexed inside your vector database (like Qdrant). Crucially, every child vector retains a metadata reference string (parent_id) pointing directly back to its source block. It makes chunks small for accurate search or make them large for accurate generation. The Parent-Child model completely breaks this trade-off by decoupling the search index from the context payload.

What are the Vector stores?

Vector Store comparison:

Feature	Qdrant	Milvus	Neo4j	Amazon OpenSearch (AWS)
Core Architecture	Monolithic Unified Engine (written in Rust)	Decoupled Distributed Microservices	Native Property Graph Database	Distributed Multi-Tenant Search Engine
Primary Data Model	Vectors + Structured Meta-Payloads	Vectors + Scalar Tables	Entities, Relationships, and Vectors	Lexical Text Documents + Vectors
Operational Effort	Very Low: Single binary, highly predictable memory usage.	High: Complex coordination of etcd, MinIO, Pulsar, and Kafka.	Medium: Straightforward setup but requires graph schema modeling.	Low to Medium: Fully managed or auto-scaling Serverless options.

Core Strengths	• Blazing fast execution • In-filtering during graph traversal • Low hardware overhead	• Scales out to billions of vectors • Isolated storage/compute layers • Rich index selection (DiskANN, ScaNN)	• Native Knowledge Graph / GraphRAG • Tracks complex data relationships • Multi-hop context tracking	• Native hybrid search (BM25 + Vector) • Seamless AWS IAM/Bedrock integration • Enterprise logging/security
Key Limitations	Not designed for deep relational data modeling or cross-table joins.	Overkill for small/mid-sized setups; high baseline infrastructure cost.	Higher vector latency and compute overhead compared to pure vector engines.	High RAM consumption; heavy index-build times under load.
Optimal Retrieval Strategies	• HNSW ANN vector search • Structural payload filtering • Multi-vector grouping	• Large-scale partition routing • Hybrid IVF + HNSW execution • Distributed segment scanning	• GraphRAG: Relational path filtering combined with vector similarity.	• Hybrid Search: Integrated Lexical BM25 + Dense Vector k-NN via RRF.
Production Fit Archetype	The Streamlined Prototyper/Agent: Ideal for real-time agentic memory and scale pools up to 100M+ vectors with minimal ops.	The Data Warehouse: Enterprise-scale operations with billions of vectors managed by dedicated platform engineering teams.	The Deep Reasoner: Advanced AI agents or GraphRAG architectures where contextual connections matter more than raw speed.	The AWS Corporate Standard: Strict cloud governance compliance requiring unified text and semantic vector search in one pool.

When to use which vector Store?

All four of these systems can handle production-grade workloads, but they fit completely different production archetypes:

- Choose Qdrant for Production if: You want low operational complexity, minimal cloud bills, fast real-time search, and your vector pool ranges from 10,000 to 100 million points. It is a favorite choice for specialized on-device or micro-agent architectures due to its efficiency.
- Choose Milvus for Production if: You have hundreds of millions to billions of vectors, dedicated data platform engineers to manage the cluster overhead, and your primary ingestion involves massive, batch-oriented data warehouse pipelines.
- Choose Neo4j for Production if: You are building an Agentic OS or advanced reasoning agent that relies on a structural Knowledge Graph. If your application logic routinely requires answering queries that link complex entities together, Neo4j is the clear choice.

- Choose Amazon OpenSearch for Production if: You are bound by strict AWS governance/security requirements, or your architecture relies heavily on legacy BM25 search logic and you want a unified platform that scales both text search and vector spaces simultaneously.

What is Document Chunking, and how to use ?

Document chunking is important process of creating vector embedding for document retrieval architecture.

LlamaIndex is a specialized data framework designed to connect Large Language Models (LLMs) with private, domain-specific, or custom data sources.

Key Characteristics & Strengths:

1. Ingestion via LlamaHub (160+ Connectors)

Before you can search data, you have to read it. LlamaIndex features LlamaHub, a massive open-source repository of data loaders. It handles data cleaning and text extraction across data format : Structural, semi structural.

2, Intelligent Data Slicing ("Nodes"): When you load a long document, LlamaIndex doesn't just break it up randomly by character count. It parses the document into a structured graph of Nodes. A Node represents a semantically meaningful chunk

3. Advanced In-Memory Indexing Strategies Unlike simple vector databases that only support flat vector spaces, LlamaIndex allows you to organize your data layout using diverse indexing structures:

- a. VectorStoreIndex: Converts nodes into dense vector embeddings for classic semantic similarity matching.
- b. SummaryIndex: Chains documents sequentially; ideal for generating global summaries over a massive timeline of events.
- c. TreeIndex: Organizes information into a hierarchical tree, enabling an LLM to traverse downwards from top-level summaries to micro-level details.
- d. KnowledgeGraphIndex: Extracts explicit entities and relationships from your text, allowing the system to run GraphRAG queries out-of-the-box.

The 5 Levels of Advanced Chunking in LlamaIndex

1. Structure-Aware Chunking (Markdown, HTML, JSON) If you give a standard character splitter a Markdown architectural spec or API doc, it might cut a table or code snippet cleanly in half. LlamaIndex avoids this with dedicated structural parsers.

2. Semantic Chunking (SemanticSplitterNodeParser) :Instead of choosing an arbitrary size limit, this splitter calculates embedding distances dynamically between adjacent sentences.
3. The Sentence Window Pattern (SentenceWindowNodeParser): This is an incredibly sharp technique for high-precision RAG. a) The Chunk: Slices text into tiny, single sentences. Only this isolated sentence is embedded and stored in the vector database. B)The Window: During retrieval, when a sentence matches, LlamaIndex automatically pulls a configurable "window" of context (e.g., 3 sentences before and 3 sentences after) and passes the complete window to the LLM.
4. Hierarchical Chunking (HierarchicalNodeParser): This natively automates the Parent-Child architecture. It parses an entire document into three separate layers concurrently: Large Parent chunks (e.g., 2048 tokens), Medium-Mid chunks (e.g., 512 tokens) Small Child leaf-nodes (e.g., 128 tokens)
5. Code Chunking (CodeSplitter) :For technical AI workloads, this module parses raw source files (Python, C++, Go) using internal language grammars (tree-sitter). It splits files by logical code blocks, such as whole classes or distinct function scopes, rather than line-breaks.